How Our System Works (Brief Guide)

POUND × Flatterie Online Booking System · for all group members

1. What the System Is

A website where customers **book a table** at POUND × Flatterie, and where the restaurant's **staff and admin manage** those bookings, tables, customers, user accounts, and old records. It is a **MERN** web app: **M**ongoDB (database), **E**xpress + **N**ode.js (server/API), **R**eact (website).

User	What they can do
Customer	Browse the website, register, log in, book a table, see and cancel their own bookings, edit their profile
Staff	Everything in the dashboard: confirm, reject, complete, edit and create bookings; manage tables; view customers; archive
Admin	Everything staff can do **+ manage user accounts** (add staff, edit, deactivate, archive accounts)

2. The Big Picture (think of a restaurant)

React website The dining room: what the user sees and clicks → **API routes** The waiter: takes the request to the right place → **Middleware** The guard: checks the login "wristband" and role → **Controllers** The kitchen: applies the rules and does the work → **MongoDB** The storage room: where all data is kept

Every action follows the same path. Example, a customer books a table:

1. The customer fills in the **booking form** (React) and clicks Book.
2. The website sends a request to the server: `POST /api/reservations`, with the login token attached.
3. The server **checks the token** (who is this?) and the **role** (are they allowed?).
4. The controller **checks the rules** (valid date? free table?) and **picks the smallest free table** that fits.
5. The booking is **saved in MongoDB** as **Pending**, and the answer goes back to the website, which shows "success".

3. Where Things Are (folders)

Folder	What's inside
`client/src/pages`	Full pages: Home, Login, Register, My Reservations, Profile, and the dashboard pages
`client/src/components`	Reusable pieces: Navbar, booking form, pop-ups, forms, menu carousel
`client/src/api`	The functions that talk to the server (one file per area)
`server/routes`	The list of URLs the server answers (the "waiter")
`server/middleware`	Login and role checks, error handling (the "guard")
`server/controllers`	The logic and rules (the "kitchen")
`server/models`	The shape of each record in MongoDB: User, Table, Reservation

`server/data` + `seed.js`	The demo data and the command that loads it

4. The Modules

Module	In short
Account Management	Register (always as a customer), log in, log out, edit profile, change password, and **forgot password** (a one-time reset link, valid for 15 minutes, shown in the server window because the email is simulated).
User Management (admin)	See all users, search and filter, add accounts (this is how staff are created), edit, activate or deactivate. Safety rules: the admin can't deactivate themselves or change their own role, and there's always at least one active admin.
Reservations (core)	Customers book online; staff manage every booking: confirm, reject, edit, complete, mark no-show, and create phone/walk-in bookings.
Tables	Add and edit tables, take a table out of service, and see a daily schedule of which table is booked in each seating.
Customers	Staff view customers' contact details and booking history (view only).
Archive	Finished bookings and deactivated accounts can be moved out of the main lists, searched, and **restored**. Nothing is deleted.
Overview	The dashboard's first page: bookings today, guests expected, pending requests, tables in service.

5. The Rules Everyone Must Know

A booking's life cycle

Pending (just booked) → **Confirmed** (staff accepted) → **Completed** (the guests dined). At Pending or Confirmed it can become **Cancelled** (cancelled by the customer, rejected by staff, or a no-show). **Completed and Cancelled are final.**

Seatings and limits

5 seatings a day, 2 hours each: **11 AM, 1 PM, 3 PM, 5 PM, 7 PM**. 1 to 10 guests. Bookings up to 30 days ahead.

No double booking

A table can have only **one Pending or Confirmed booking per seating**. Customers don't choose tables: the system gives the **smallest free table that fits** the group. Staff can choose, but only free tables are offered.

Archive, not delete

Only **Completed or Cancelled** bookings and **deactivated** accounts can be archived. Archived records are hidden from the main lists and can be restored anytime.

Security

Passwords are **hashed** (scrambled, never stored as typed). After login, the server gives a **token** (JWT), like a wristband, that the website shows with every request. The server checks the token and the role before doing anything.

6. How to Run It

1. Terminal 1: `cd server` → `npm run dev` → wait for "MongoDB connected".
2. Terminal 2: `cd client` → `npm run dev` → open `http://localhost:5173`.
3. `npm run seed` (in `server`) resets the database to the demo data: **12 users, 40 bookings, 26 completed**. Tell the group first, because the database is shared.

Role	Email	Password
Admin	`giuz.montano@gmail.com`	`Admin@123`
Staff	`rene.baterbonia@gmail.com` / `marco.delosreyes@gmail.com`	`Staff@123`
Customer	`maria.santos@gmail.com` (and 8 others)	`Customer@123`

7. Ten Words to Know

Frontend	The website part (React), runs in the browser
Backend	The server part (Express + Node.js), holds the rules
API	The list of URLs the website uses to ask the server for things
Route	One URL + method, e.g. `POST /api/reservations`
Middleware	Checks that run before the main logic (login, role)
Controller	The function that does the actual work and applies the rules
Model	The shape and rules of one kind of record (User, Table, Reservation)
Collection	A group of records in MongoDB (like a table in other databases)
Token (JWT)	Proof of login that the website sends with each request
Seed	Loading the prepared demo data into the database